

CONTEXT

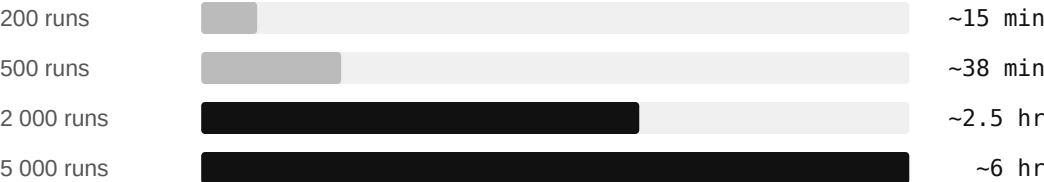
Data generation could not keep up

MATLAB produced training data one run at a time across 221 depth nodes and 600 hourly timesteps

THE BOTTLENECK

Each simulation run solves Richards' equation across 221 depth nodes and 600 hourly timesteps. Strategy 3 needs thousands of varied runs to prove the PGML surrogate generalises across heap sites. Serial execution makes this prohibitively slow, with no crash recovery.

SERIAL GENERATION COST



A crash at run 1 800 of 2 000 means starting over. Strategy 3 demands a better approach.

A three-stage data generation plan

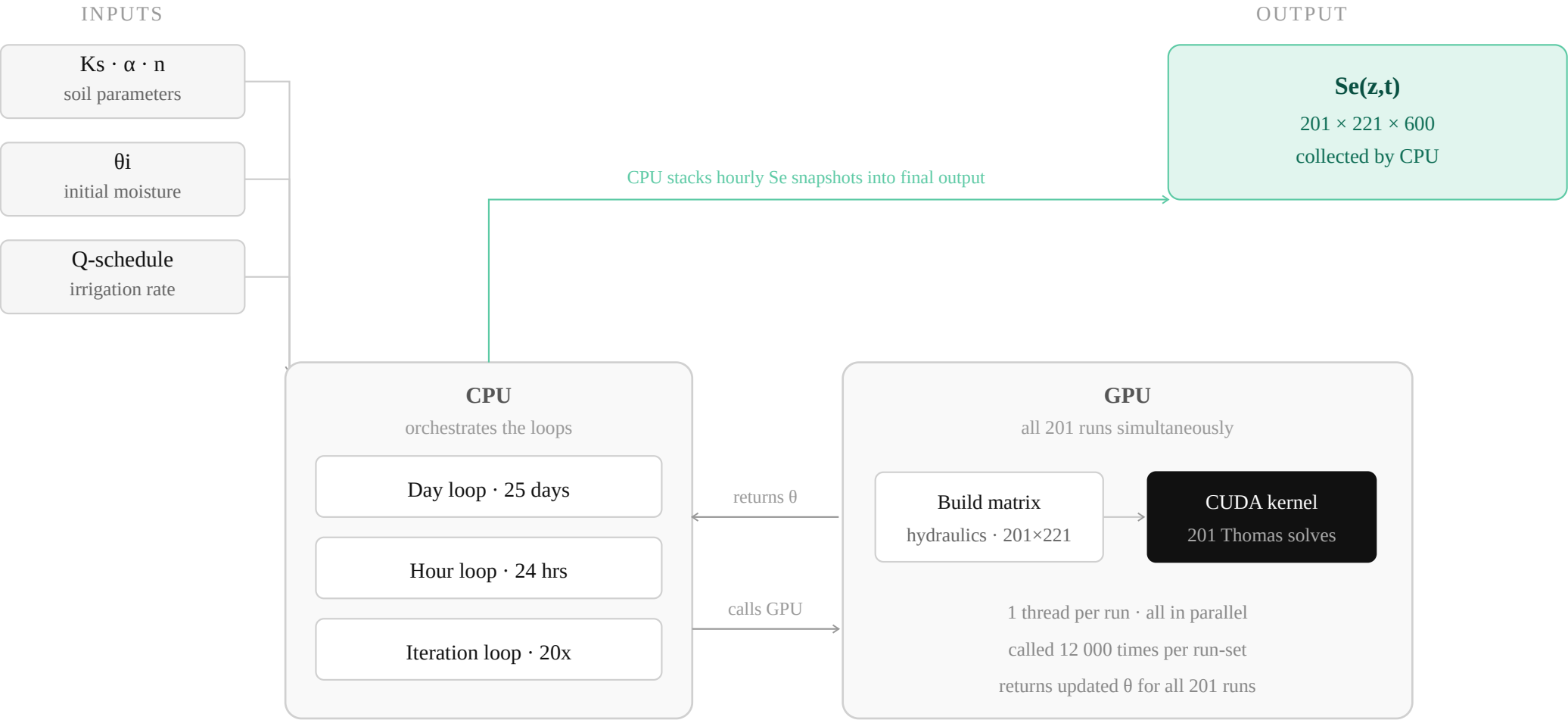
Each stage builds on the last. Strategy 3 is the target for a universal PGML surrogate.

STRATEGY	WHAT VARIES	WHAT IS FIXED	GOAL
1	Q-schedule	Ks, α , n, θ_i at Cariaga nominal	Pipeline working end-to-end
1+	Q-schedule, θ_i	Ks, α , n at Cariaga nominal	Add initial moisture variation
2+	Q (structured), θ_i	Ks, α , n at Cariaga nominal	Add control-informed schedules
3 ★	Ks, α , n, θ_i , Q-schedule	Column geometry, θ_s , θ_r	Generalise surrogate across heap sites

★ Strategy 3 parameters sampled via Sobol sequences across a 6-dimensional scalar space. Q-schedule parameterised as mean + spread. Pilot of 64 runs validated today.

Parameters feed the GPU. The physics stays on CPU.

Each run is independent. The GPU processes all of them simultaneously.



Rebuilt the simulator in Python, verified to machine precision

Every MATLAB file ported faithfully. Correctness confirmed before any optimisation.

MATLAB TO PYTHON MAPPING

MATLAB		PYTHON	ROLE
S/h/K/D/d0dh_theta.m	→	hydraulics.py	Five VGM relations, vectorised
theta_m1_theta.m	→	ghost_node.py	Top boundary condition
Pred_correct_b.m / Pred_correct.m	→	pred_correct.py	Matrix assembly and solver
Sim_column.m / columna.m	→	sim_column.py	Serial driver, trusted reference
(new)		thomas_kernel.py	Custom CUDA kernel, 170x faster
(new)		sim_gpu.py	Batched GPU driver

VERIFICATION RESULT

201 runs compared	1e-13 worst Se error
---	--

Each run re-simulated using its stored Q-schedule and compared against MATLAB Se_out across all 221 x 600 points. The error is effectively zero — floating-point noise from arithmetic ordering between MATLAB and NumPy, not a physics deviation.

We let data pick the solver

A timed head-to-head at two batch sizes determined the right approach.

WHAT WAS COMPARED

Thomas algorithm

Specialist tridiagonal solver. Knows the matrix has exactly three diagonals. Minimal arithmetic, no matrix stored. One GPU thread per run.

Sparse library solver

General-purpose. Assembles a large block-diagonal matrix from all runs and calls the library. Pre-written, less code to own.

MEASURED ON RTX 5060, PER SOLVE

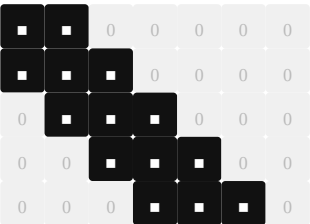
B = 200 RUNS



B = 2 000 RUNS

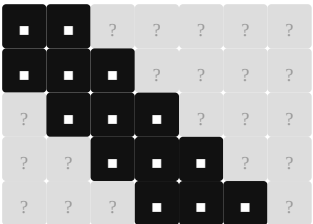


TRIDIAGONAL — Thomas



zeros confirmed · Thomas skips them

GENERAL SPARSE SOLVER



checks every cell · unnecessary work at scale

VS
29x slower
at B=2000

A custom CUDA kernel solved the performance problem

The first GPU attempt was still slow. One kernel launch per solve fixed it.

WHAT WENT WRONG FIRST

The Thomas solver looped over 221 nodes in Python. Each node-step launched a separate tiny GPU kernel — over 5 million launches per run, each doing almost nothing. The GPU spent most of its time waiting, not computing. Result: 500 runs in over 5 minutes, slower than expected.

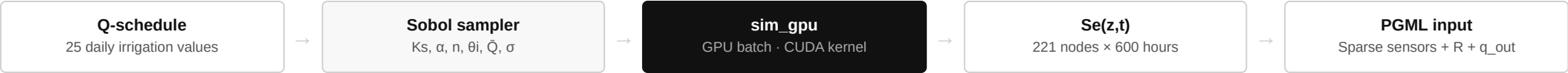
THE FIX

A custom CUDA kernel written in C, called via CuPy's RawKernel API. One GPU thread per run. That thread executes the entire 221-node Thomas sweep internally without returning to Python. One launch replaces millions. The original 201 MATLAB runs took 15 minutes serially. The fixed GPU version does 500 runs in 39 seconds.

~15 min MATLAB serial · 201 runs	5 min+ GPU first attempt · 500 runs	170x kernel speedup · RTX 5060	39 s GPU with CUDA kernel · 500 runs
--	---	--	--

500 runs in 39 seconds, verified against MATLAB

The full data generation pipeline from Q-schedule to Se surface, every step confirmed.



Serial Python vs MATLAB

201/201 runs passed. Worst error 1.3e-13 — effectively zero. Floating-point noise from arithmetic ordering, not a physics deviation.

GPU batch vs MATLAB

201/201 runs passed. Same worst error. Batched GPU output is identical to serial Python and original MATLAB.

Strategy 3 pilot

64 Sobol runs with varied Ks, α, n, θi. Data generation pipeline validated for Strategy 3 conditions.

Three items to resolve before scaling

Findings from the pilot that require further investigation.

ITEM 1

Numerical stability at extreme parameters

A subset of extreme soil parameter combinations produced NaN in the VGM solver during the pilot. Low values of n are a suspected contributor due to the conductivity exponent $\delta = 3 + 2/(mn)$ growing large, but the root cause needs verification across the full parameter space.

Investigate stability boundaries and establish safe sampling ranges before scaling to 2 000+ runs.

ITEM 2

Schedule representation

Current pilot uses mean + spread to generate random-walk schedules. A physically motivated alternative is a setpoint-tracking schedule: irrigation ramps until a target Se is reached, then holds. This better reflects real NMPC operation and may improve surrogate generalisation.

Assess whether setpoint-tracking schedules should be included alongside random walks.

ITEM 3

Minimum dataset size

Sobol sequences are extensible: 256 pilot runs are the first 256 of a 2 048-run dataset at no extra cost. A convergence study — tracking surrogate accuracy as training set size grows from 256 to 2 048 — will identify where accuracy plateaus and additional runs add no value.

Run the convergence study to determine the minimum sufficient dataset size.